객체지향 프로그래밍 기초

11. 포인터 II

참조자(참조변수)

- “변수는 할당된 메모리 공간에 붙여진 이름이다.
- 그 이름을 통해서 해당 메모리 공간에 접근이 가능하다.”
- “할당된 하나의 메모리 공간에 둘 이상의 이름을 부여할 수 있는가?”

참조자

```
int main()
{
    int   mycat = 4;
    int   &darong  =  mycat;
    cout << “mycat = “ << mycat;
    cout << “ , darong = “ << darong;

    darong++;
    cout << “mycat = “ << mycat;
    cout << “ , darong = “ << darong;

    cout << “mycat의 주소 = “ << &mycat;
    cout << “ , darong의 주소 = “ << &darong <<endl;
    return 0;
}
```

참조자 정의

- 함수의 매개변수로 사용하는 참조(&)는 함수의 인수에 대한 다른 이름(별칭, alias)이다.
- 참조자는 새로운 공간을 할당하지 않고 이름만 생기며, 참조자에 할당된 변수를 공유한다.
- 참조자는 한 번 생성한 후에는 다시 할당할 수 없다.(NULL이 될 수 없다.)
- 참조자의 주된 용도는 함수의 매개변수에 사용하는 것이다.

참조자 생성

```
int rats;
int & mouse = rats;
```

- &가 주소 연산자가 아니라, 데이터형 식별자의 일부로 사용된 것이다.
- int & 는 int 에 대한 참조를 의미한다.
- rats와 mouse는 서로 바꾸어 사용할 수 있다.
- rats와 mouse는 같은 값과 같은 메모리 위치를 참조한다.

참조자 생성

참조자는 무조건 선언과 동시에 변수를 참조해야 한다.

```
int &ref = 20;
int &ref;
int &ref = NULL;
```

X

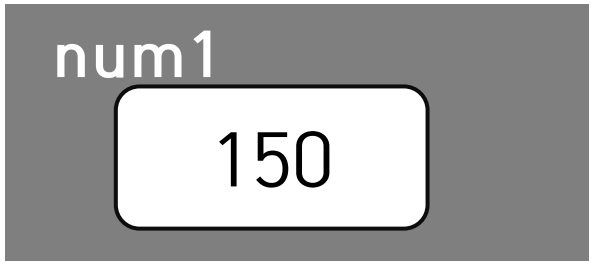
참조자

```
int main()
{
    int num1 = 150;
    int &num2 = num1;

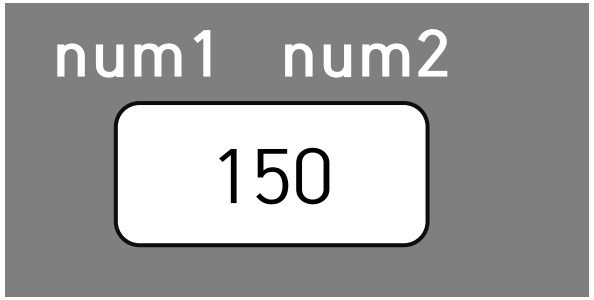
    num2 = 3047;
    cout << “num1: “ << num1 << endl;
    cout << “num2: “ << num1 << endl

    cout << “&num1: “ << &num1 << endl;
    cout << “&num2: “ << &num2 << endl;
    return 0;
}
```

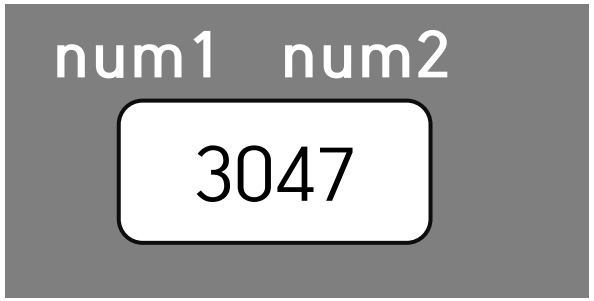
int num1 = 150



int &num2 = num1;



num2 = 3047



참조자를 이용한 매개변수 : Call by Reference

```
void Swap(int &ref1, int &ref2);

int main()
{
    int val1 = 10;
    int val2 = 20;
    Swap(val1, val2);
    return 0;
}
void Swap(int &ref1, int &ref2)
{
    int temp = ref1;
    ref1 = ref2;
    ref2 = temp;
}
```

void Swap(10,20);
-> 오류가 발생하는 이유는?

참조자와 포인터

종류	포인터	참조자
NULL 검사	NULL이 될 수도 있기 때문에 NULL인지 확인한다.	NULL이 될 수 없으므로 NULL검사가 필요없다.
객체주소	객체주소를 가리키지만 가리키는 것이 없으면 0값을 갖는다.	항상 어떤 객체를 가리켜서 참조한다.
함수호출방법	&포인터명을 사용하여 인자전달	참조명으로 인자전달 (변수와 동일하게 사용)

- **참조할 대상이 항상 존재하면 참조자를 사용하고 그 대상이 존재하지 않을 수도 있으면 포인터를 사용한다.**
- **참조할 객체를 미리 알고 있거나 다른 객체로 바꾸어 참조할 일이 없으면 참조자를 사용하고 이외에는 포인터를 사용한다.**

[예제1]

(1) 참조자를 이용하여 인자로 전달된 int형 변수의 값을 1씩 증가시키는 함수를 정의하시오.

(2) 인자로 전달된 int형 변수의 부호를 바꾸는 함수를 정의하시오.

const 참조자

참조자를 사용할 경우 값이 변경될 수도 있는데 이를 막기 위해 const를 사용

```
int main()
{
    int num = 24;
    HappyFunc(num);
    cout << num << endl;
    .. . .
}
void HappyFunc(int &ref)
{
     ref = 10;
 }
```

```
void HappyFunc(const int &ref) { .. }
```

"함수 HappyFunc 내에서 참조자 ref를 이용한 값의 변경은 하지 않겠다."

반환형이 참조형인 경우

```
int& RefFuncOne(int &ref)
{
    ref++;
    return ref;
}
```

VS

```
int RefFuncTwo(int &ref)
{
    ref++;
    return ref;
}
```

```
num1 = 10;
int &num2 = RefFuncOne(num1);
```

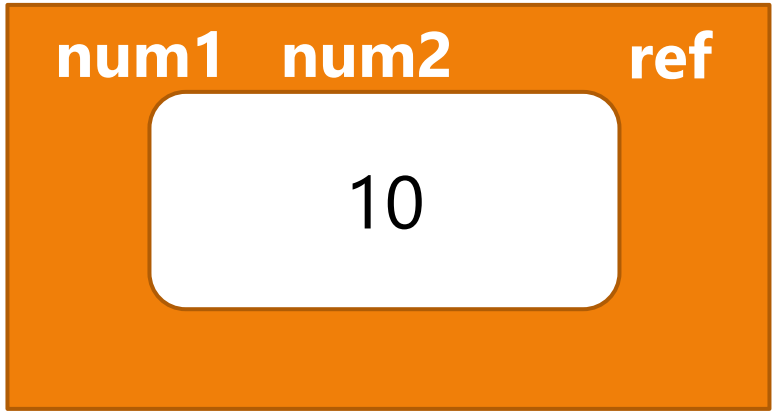
```
num = 10;
int num2 = RefFuncTwo(num1);
```

num1과 num2의 값은?

```
#include <iostream>
using namespace std;

int& RefFuncOne(int &ref)
{
    ref++;
    return ref;
}

int main()
{
    int num1 = 10;
    int &num2 = RefFuncOne(num1);
    num1+=1;
    num2+=100;
    cout << num1 << " " << num2 << endl;
    return 0;
}
```




```
int num1=10;
int &ref=num1;
int &num2=ref;
```

112, 112

```
include <iostream>
using namespace std;

int RefFuncOne(int &ref)
{
    ref++;
    return ref;
}

int main()
{
    int num1 = 10;
    int num2 = RefFuncOne(num1);
    num1+=1;
    num2+=100;
    cout << num1 << " " << num2 << endl;
    return 0;
}
```





12, 111

```
#include <iostream>
using namespace std;

int RefFuncOne(int &ref)
{
	ref++;
	return ref;
}
int main()
{
	int num1 = 10;
	int num2 = RefFuncOne(num1);
	num1+=1;
	num2+=100;
	cout << num1 << "\t" << num2 << endl;
	return 0;
}
```



int &num2 = RefFuncOne(num1); X

왜 안되는가?

⇒RefFuncOne()함수의 return값이 int형이기 때문

프로그램의 오류는?

```
int& RefFuncOne(int n);

int main()
{
	int num1 = 10;
	int &num2 = RefFuncOne(num1);
	num1+=1;
	num2+=100;
	cout << num1 << "\t" << num2 << endl;
	return 0;
}
```

```
int& RefFuncOne(int n)
{
	int num = 0;
	num += n;
	return num;
}
```

함수가 반환이 되면 지역변수 num은 소멸됨
지역변수를 참조형으로 반환하면 안됨

11강

포인터의 포인터

포인터의 포인터(이중포인터)

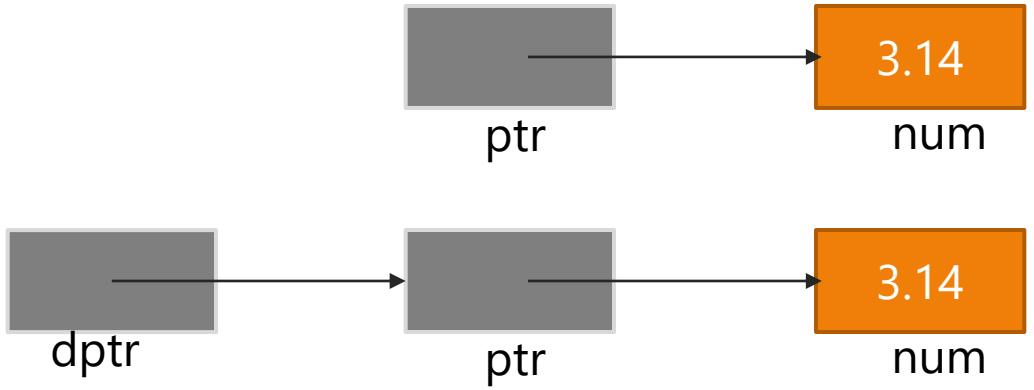
- 포인터의 포인터(이중포인터)는 포인터 변수를 가리키는 또 다른 포인터 변수를 뜻한다.
- 이중 포인터 변수의 선언 방법

```
int ** dptr;
```

포인터의 포인터(이중포인터)

```
int main()
{
    double num=3.14;
    double * ptr = &num;

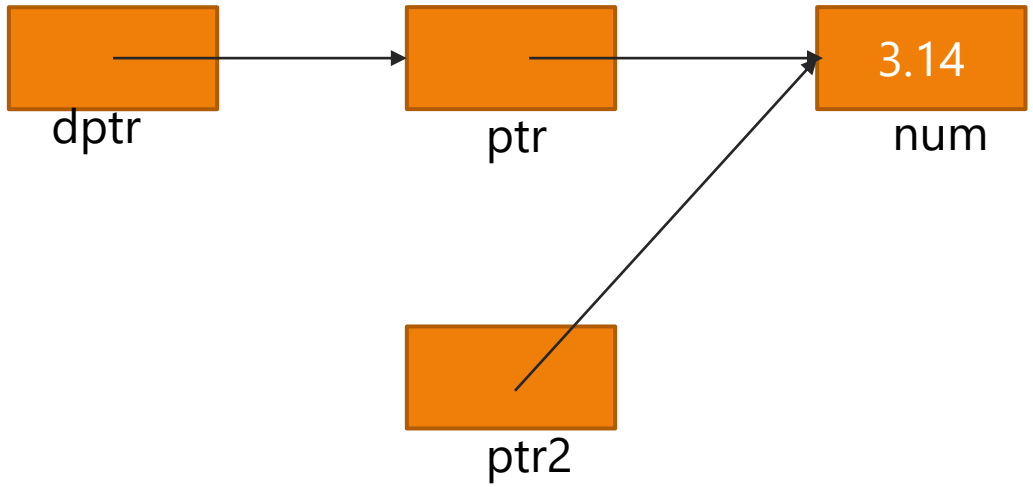
    double ** dptr = &ptr;
    . . .
}
```




```
*dptr=. . .;     // *ptr은 포인터변수 ptr을 의미
**dptr=. . .;    // 변수 num을 의미
```

```
int main()
{
    double num = 3.14;
    double *ptr = &num;
    double **dptr = &ptr;
    double *ptr2;

    cout << ptr << " " << *dptr << endl;
    cout << num << " " << **dptr << endl;
    ptr2 = *dptr;
    *ptr2 = 19.3;
    cout << num << " " << **dptr;
    return 0;
}
```



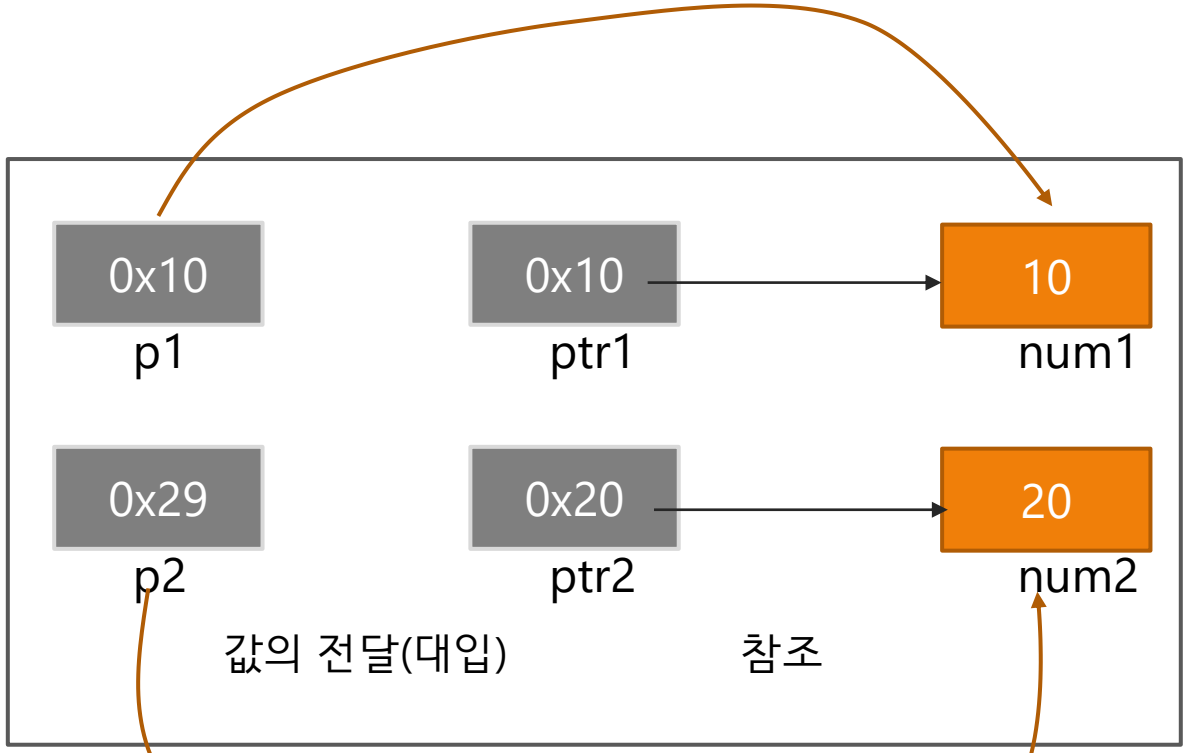
포인터 변수 대상의 call-by-reference

```
void SwapIntPtr(int *p1, int *p2)
{
    int *temp = p1;
    p1 = p2;
    p2 = temp;
}

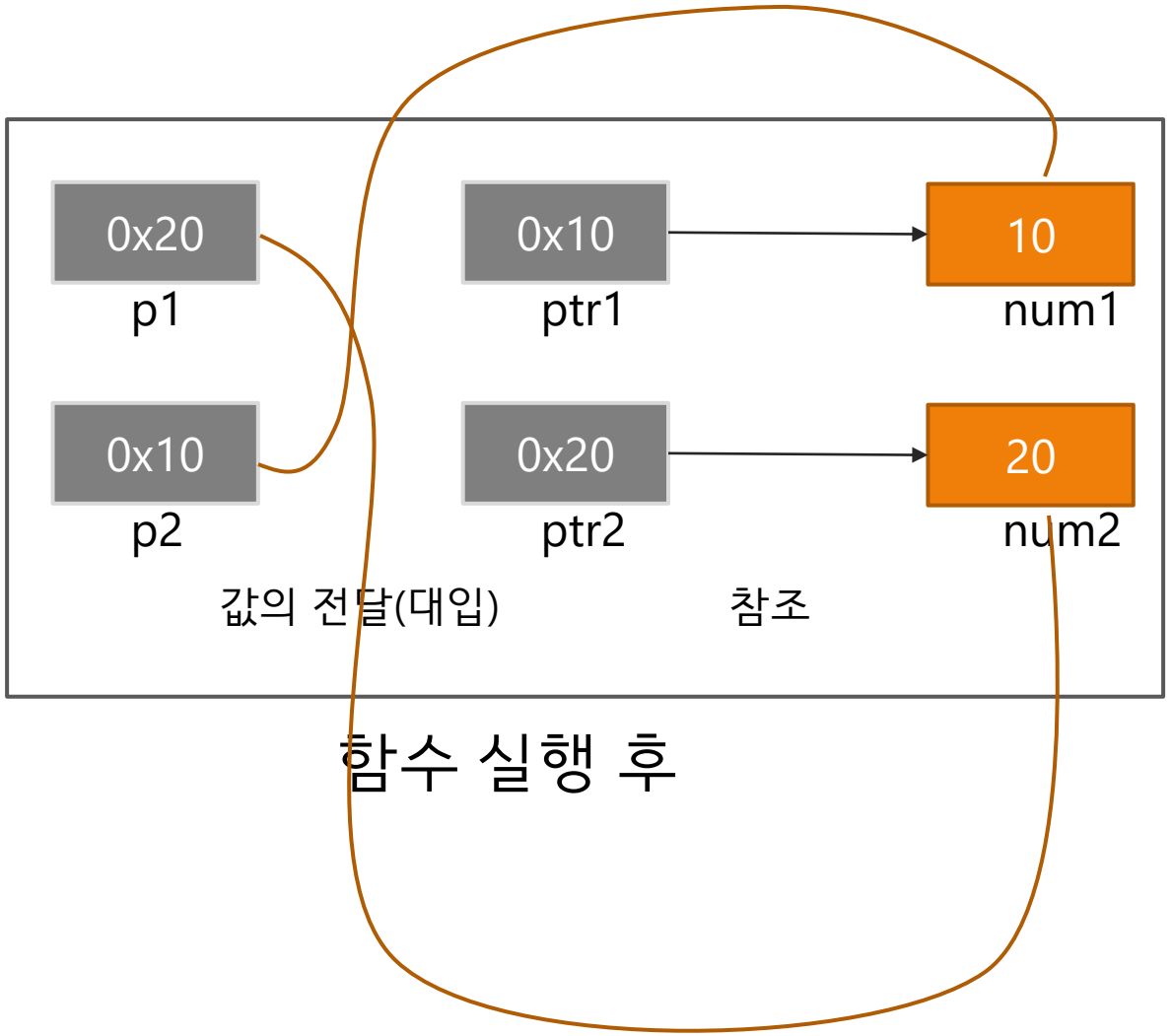
int main()
{
    int num1 = 10, num2 = 20;
    int *ptr1, *ptr2;
    ptr1 = &num1; ptr2 = &num2;

    SwapIntPtr(ptr1, ptr2);
    cout << *ptr1 << *ptr2 << endl;
    return 0;
}
```

=> 실행결과는?



매개변수 전달 직후



함수 실행 후

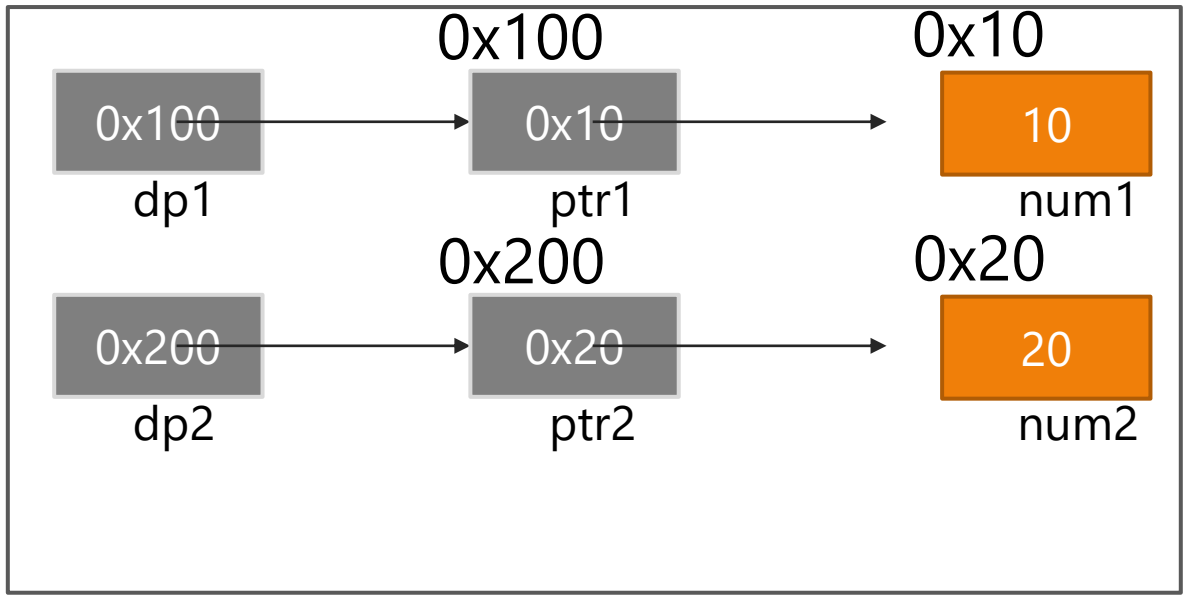
포인터 변수 대상의 call-by-reference

```
void SwapIntPtr(int **dp1, int **dp2)
{
	int *temp = *dp1;
	*dp1 = *dp2;
	*dp2 = temp;
}

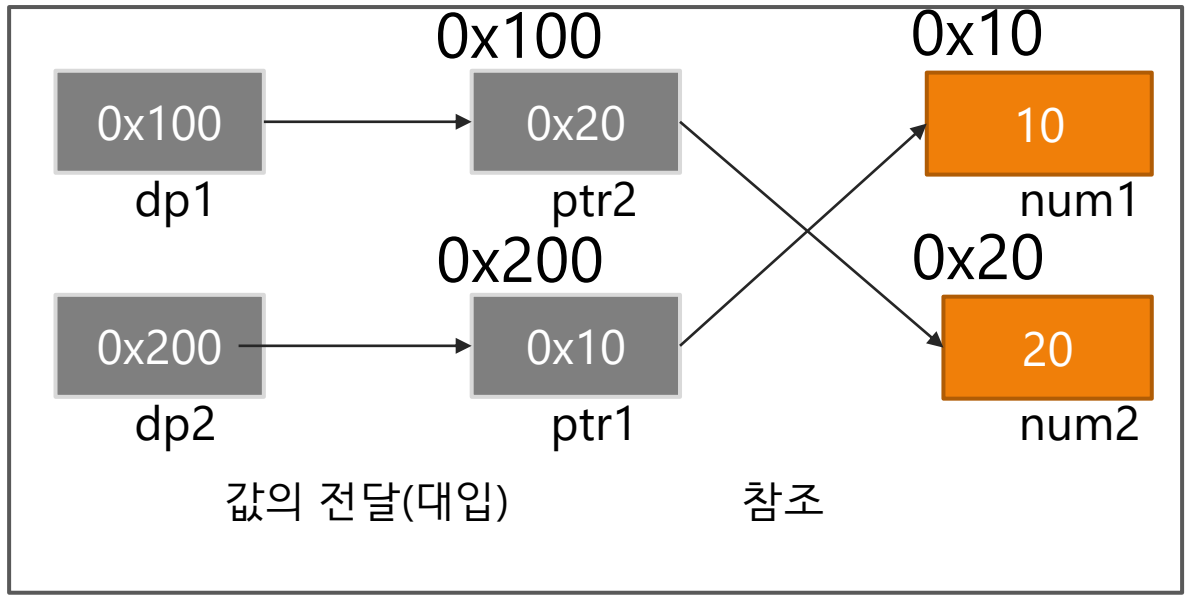
int main()
{
	int num1 = 10, num2 = 20;
	int *ptr1, *ptr2;
	ptr1 = &num1; ptr2 = &num2;

	SwapIntPtr(&ptr1, &ptr2);
	cout << *ptr1 << *ptr2 << endl;
	return 0;
}
```

=> 실행결과는?



매개변수 전달 직후



함수 실행 후

[예제1]

int형 배열에 5개의 랜덤한 숫자가 들어있다. 이 배열의 숫자들 중 최대값이 저장된 배열요소의 주소값과 최소값이 저장된 배열요소의 주소값을 구하는 함수 MaxAndMin()을 작성하시오.

```
void MaxAndMin(int *arr, int size, int **maxPtr, int **minPtr);
```

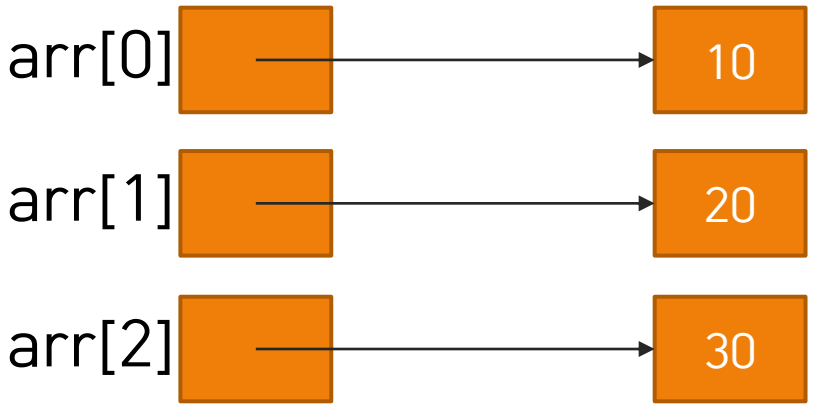
포인터 배열

int * arr[4];

배열 arr의 원소 4개는 int를 가리키는 포인터를 저장하고 있다.

```
int main()
{
    int num1 = 10, num2 = 20, num3 = 30;
    int *arr[3] = { &num1, &num2, &num3 };

    cout << *arr[0] << endl;
    cout << *arr[1] << endl;
    cout << *arr[2] << endl;
    return 0;
}
```



포인터배열과 이중 포인터

```
int main()
{
    int num1=10, num2=20, num3=30;
    int *ptr1=&num1;
    int *ptr2=&num2;
    int *ptr3=&num3;

    int *ptrArr[]={ptr1, ptr2, ptr3};
    int **dptr=ptrArr;

    cout << *(ptrArr[0]) << " " << *(ptrArr[1])<< " "<< *(ptrArr[2]);
    cout << *(dptr[0]) << " " << *(dptr[1]) << " " << *(dptr[2]);
    return 0;
}
```

포인터배열과 이중포인터

```
int * parr[20];
int ** dparr;
```

포인터배열은 이중포인터이다.

2차원 배열 포인터(배열 포인터 변수)

int a[4][3];



- a는 첫번째 요소를 가리키면서 배열 전체를 의미한다.
- a[0]는 첫번째 요소를 가리키되 1행만을 의미한다.

2차원 배열 포인터(배열 포인터 변수)

int a[4][3]; = int (*ptr)[3];



- a는 첫번째 요소를 가리키면서 배열 전체를 의미한다.
- a[0]는 첫번째 요소를 가리키되 1행만을 의미한다.

2차원 배열 포인터

- **가리키는 대상은 무엇인가?**
- **배열이름(포인터)을 대상으로 값을 1증가 시 실제로 얼마가 증가하는가?**

int arr[4][3];

⇒실제로 가리키는 대상은 int형 변수

⇒포인터 연산시 sizeof(int) X 3의 크기단위로 값이 증가

⇒ int (*ptr)[3];

```
int main()
{
    int arr2d[3][3];
    cout << "arr2d: " << arr2d << endl;
    cout << "arr2d[0]: " << arr2d[0] << endl;
    cout << "&arr2d[0][0]: " << &arr2d[0][0] << endl;

    cout << "arr2d[1]: " << arr2d[1] << endl;
    cout << "&arr2d[1][0]: " << &arr2d[1][0] << endl;

    cout << "&arr2d[2]: " << arr2d[2] << endl;
    cout << "&arr2d[2][0]: " << &arr2d[2][0] << endl;

    cout << "sizeof(arr2d): " << sizeof(arr2d) << endl;
    cout << "sizeof(arr2d[0]): " << sizeof(arr2d[0]) << endl;
    return 0;
}
```

```
int main()
{
	int arr1[2][2] = { {1,2},{2,3} };
	int arr2[3][2] = { {1,2},{2,3},{3,4} };
	int(*ptr)[2];
	int i;
	ptr = arr1;
	for (i = 0; i < 2; i++) {
		cout << ptr[i][0] << "\t" << ptr[i][1];
		cout << endl;
	}
	cout << endl;

	ptr = arr2;
	for (i = 0; i < 3; i++) {
		cout << ptr[i][0] << "\t" << ptr[i][1];
		cout << endl;
	}
	cout << endl;
	return 0;
}
```

2차원 배열 포인터 vs 포인터배열

- int * whoA[4];　　//int형 포인터변수로 이루어진 포인터 배열
- int (*whoB)[4];　//가로길이가 4인 int형 2차원 배열을 가리키는 포인터 변수

　　= int (*parr1)[7] = int parr1[][7]

2차원 배열 포인터 vs 포인터배열

```
int main()
{
	int num1 = 10 , num2 = 20, num3 = 30;
	int arr2d[2][3] = { 1,2,3,4,5,6 };
	int i, j;

	int *whoA[3] = { &num1, &num2, &num3 };
	int(*whoB)[3] = arr2d;

	cout << *whoA[0] << "\t" << *whoA[1] << "\t" << *whoA[2] << endl;
	for (i = 0; i < 2; i++)
	{
		for (j = 0; j < 3; j++)
		cout << whoB[i][j] << "\t";
	}
	cout << endl;
	return 0;
}
```

[예제]

다음 2차원 배열의 합을 구하는 함수를 작성하시오.(함수의 인자로 배열포인터를 사용)

```
int main()
{
        int arr1[2][4]={1,2,3,4,5,6,7,8};
        int sum = arr2DSum(arr1, sizeof(arr1)/sizeof(arr1[0]));
        cout << sum << endl;
        return 0;
}
```

```
int arr2DSum(int (*arr)[4], int col)
{
        int i, j, sum=0;
        for (i=0;i<col;i++) {
                for(j=0;j<4;j++)
                        sum+=arr[i][j];
        }
        return sum;
}
```

```
int arr2DSum(int arr[][4], int col) O
```

11강

함수 포인터와 void 포인터

void형 포인터

형(Type)이 존재하지 않는 포인터 변수

```
void * ptr;
```

포인터 변수에는 어떤 변수의 값도 담을 수 있다.

```
void SimpleFunc()
{

}

int main()
{
	int num = 20;
	void *ptr;
	ptr = &num;
	cout << ptr << endl;
	ptr = SimpleFunc;
	cout << ptr << endl;
	return 0;
}
```

void형 포인터

void형 포인터 변수는 아무런 포인터 연산도 하지 못하며, 값의 변경이나 참조도 불가능하다.

```
int main()
{
	int num = 20;
	void *ptr;
	ptr = &num;
	cout << *ptr << endl;
	return 0;
}
```

함수 포인터

- 프로그램을 실행하면 프로그램의 기계어로 번역된 모든 부분이 메모리 영역에 저장되고 실행된다. 함수의 시작주소가 함수의 실행코드가 존재하는 메모리 영역의 시작 주소값이다.
- 함수의 시작주소를 갖는 포인터를 사용하면 해당 함수를 호출할 수 있다.

```
int add(int x, int y);

int main()
{
	int(*fp)(int x, int y);
	int a = 3, b = 7, ans;

	fp = add;
	ans = fp(a, b);

	cout << "3 + 7 = " << ans << endl;
	return 0;
}

int add(int x, int y)
{
	return x + y;
}
```